Шпаргалка архитектора: Мультиагентные системы

> **Основано на лекции:** Мультиагентные системы: границы, оркестрация, паттерны (Николай Барышников)
> **Фокус:** Проектирование архитектуры, управление контекстом, паттерны взаимодействия.

1. Декомпозиция и границы (Когда нужен субагент)

Основная цель выделения субагента — сохранение качества результата. Декомпозируйте задачу, если требуется:

- **Отдельный контекст:** Самая частая причина. Чем меньше контекста в промпте, тем выше качество ответа. Это предотвращает попадание агента в «тупую зону» контекста.
- **Разные полномочия:** Разграничение доменных зон (например, только этот субагент имеет право регистрировать обращения или обновлять базу).
- **Независимая проверка:** Оценка результата с «чистым» контекстом (например, агент-скептик или Oracle).

Базовая декомпозиция обращения: 1) Понять и найти (Research — самая сложная задача), 2) Проверить и решить, 3) Подтвердить и ответить.

2. Контракт делегирования и Guardrails

Не пишите просто «сделай что-то» или «ты сеньор-разработчик». Рабочий контракт должен содержать:

- **Цель и входные данные:** Описание текущей ситуации и доступных инструментов.
- **Доступы и ограничения:** Что строго запрещено (например, «не выходи за лимит токенов», «не пиши тесты — они сейчас избыточны»).
- **Бюджет:** Лимит на количество повторов, вызовов и исправлений. Не ставьте жесткий лимит, обрывающий процесс (агент потеряет контекст). Лучше правило: «если вышел за бюджет, заверши работу в ближайшее время».
- **Условия эскалации:** Как и когда нужно позвать скептика (Oracle) или человека.

Guardrails — это набор правил, которые вставляются в *системный промпт* и передаются при каждом запросе, поэтому они не «тонут» в истории диалога. Подгружайте их динамически (например, из собственного MCP) при старте проекта.

3. Оркестрация: Внутреннее устройство

Если приоткрыть «черный ящик» оркестратора, внутри всегда работают два контура управления:

Task Ledger (Data Plane)

Список задач, связанных с изначальной целью пользователя. Формируется после цикла размышлений как *план достижения цели*. Подвержен забыванию при переполнении контекста.

Progress Ledger (Control Plane)

План, контролирующий сам процесс работы мультиагентной системы: в каком статусе агенты, как часто их опрашивать, как работать с общей памятью и когда запускать проверки.

4. Кто принимает решения: Модель, Runtime и Человек

Уровень	Зона ответственности	Ограничения
Модель	Интерпретация происходящего, генерация следующих шагов, маршрутизация.	Склонна к галлюцинациям. **Нельзя доверять проверки прав** (даже 4 уровня проверок-цензоров в промптах не гарантируют безопасность).
Runtime	ACL (контроль доступов), лимиты, таймеры, идемпотентность, RegExp-проверки.	Дискретный код (ваша ОС, скрипты). Единственный надежный способ контроля безопасности.
Человек	Юридическая ответственность, финансы, критические исключения, негативная тональность.	Является «бутылочным горлышком», но необходим для критических узлов.

5. Инженерные инсайты

- **Code Mode:** Работайте со строгой структурой данных (JSON) и кодом. Поиск конкретных слов лучше делать через `ripgrep`. Точные преобразования статусов делайте кодом, а не моделью. Это экономит контекст в сотни раз.
- **Аппаратные ограничения:** Если запустить параллельно 500 агентов, узким местом станет **диск** (операции ввода-вывода), а затем оперативная память, а не CPU.
- **Тайм-ауты и восстановление:** Тайм-аут (сети, чтения файла) — это рассинхрон во времени. Для возобновления скажите модели: *«прочитай фактический статус и продолжай выполнение моего плана»*. Выполняйте повторно только идемпотентные операции.

- **Системный промпт:** Всегда проверяйте системный промпт агента «из коробки» — часто там много мусора. Инструменты вроде **Pi** позволяют тонко настраивать системный промпт и профили субагентов (effort + модель).

6. Паттерны взаимодействия

Паттерн	Применение и механика
Иерархия (Дерево)	Рекурсивная декомпозиция. Отлично подходит для Research и анализа больших (legacy) проектов (субагенты запускаются под конкретные файлы/функции).
Swarm (Рой)	Синхронное общение агентов напрямую (без оркестратора). Эффективно для коллективной работы над большим общим ресурсом (например, Data Lake).
Цикл: Генератор → Критик	Базовый подход для достижения высокого качества. Для создания отличного плана обычно требуется **от 6 до 10 итераций** с независимым ревьюером (Oracle).
Fan-out / Fan-in	Асинхронный параллельный запуск агентов для независимого анализа (например, шерстить разные БД) со сборкой результатов в основном потоке.
Handoff vs HITL	**Handoff:** Агент автономно работает в фоне, пока оператор/другой агент продолжает диалог. **Human-in-the-loop (HITL):** Остановка процесса до решения человека (через Telegram-бот/MCP).